KU LEUVEN

FACULTEIT INDUSTRIELE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Geavanceerde Computerarchitectuur

Report 3
MEI-ecfs

Charlotte BERT
Sander HAUSTRAETE

Academiejaar 2025 - 2026

1 Context

In dit labo onderzoeken we de invloed van GPU-threading en thread divergence op beeldverwerking. In het eerste deel wordt een afbeelding geïnverteerd en wordt geanalyseerd hoe het aantal threads de uitvoeringstijd beïnvloedt. In het tweede deel wordt de rode kleurcomponent verwerkt met een 1D-Gaussian blur, waardoor thread divergence ontstaat en wordt gekeken hoe dit de prestaties beïnvloedt. Het doel is inzicht te krijgen in de relatie tussen kernelontwerp en efficiëntie op de GPU.

2 Methodologie

Eerst begonnen we met het schrijven van de `invert_image` kernel (zie Codefragment 1), die voor elke kleurcomponent (RGB) de bewerking $x = 255 - x$ uitvoert. Hierbij werd één thread per pixelwaarde gebruikt. Vervolgens werd deze kernel uitgebreid met *striding* zodat één thread meerdere pixelwaarden verwerkt. Op die manier konden verschillende verhoudingen tussen het aantal threads en het aantal elementen (pixelwaarden) gebruikt worden.

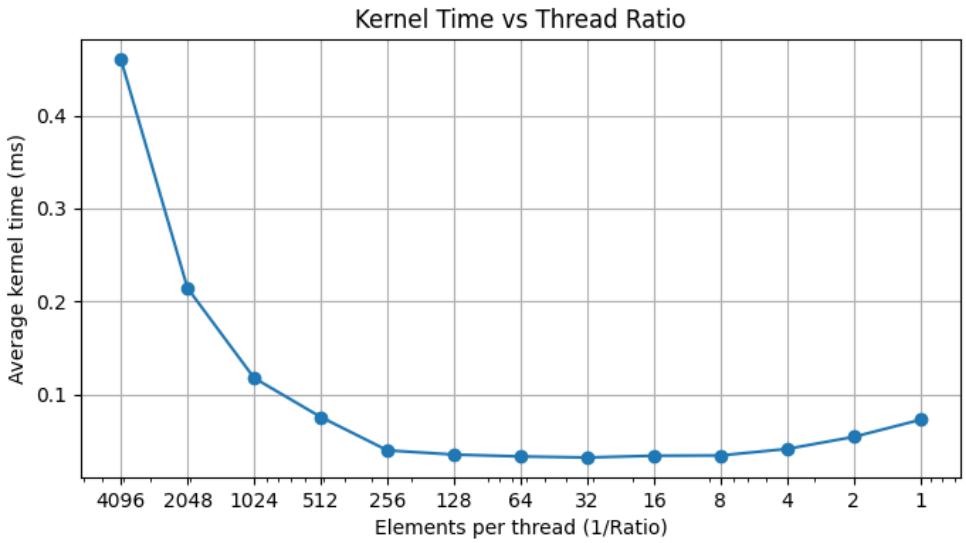
Om het effect van thread divergence te analyseren werd een kernel geschreven die 1D gaussiaanse blur toepast op de rode kleurcomponent terwijl de groene en blauwe componenten geïnverteerd blijven. Er werden twee varianten ontwikkeld. De eerste versie verwerkt de beelddata volgens `RGBRGB` en laat threads binnen eenzelfde warp verschillende uitvoeringspaden volgen. De tweede versie gebruikt een herordening van de beelddata volgens `RR...RGG...GBB...B`, waarbij alle threads binnen eenzelfde warp dezelfde berekeningen uitvoeren en thread divergence vermeden wordt.

3 Observatie/ resultaat

3.1 Deel 1: warps

```
# elements: 3686400
threadsPerBlock: 256, repeats (sample size): 500
ratio(threads/elements) blocks  threads avg_ms
1:4096                  4       1024    0.471902
1:2048                  8       2048    0.212744
1:1024                  15      3840    0.116056
1:512                   29      7424    0.0652327
1:256                   57      14592   0.0396207
1:128                   113     28928   0.0371446
1:64                    225     57600   0.0312586
1:32                    450     115200  0.0292681
1:16                    900     230400  0.0370668
1:8                     1800    460800  0.0336487
1:4                     3600    921600  0.0418243
1:2                     7200    1843200 0.0462684
1:1                     14400   3686400 0.0674222
```

Figuur 1: Terminal output van uitvoeringstijden van verschillende ratio's



Figuur 2: Grafiek uitvoeringstijd vs thread ratio

Zoals te zien is op figuur 1 en figuur 2, waar de gemiddelde uitvoeringstijd van 500 samples wordt getoond voor verschillende ratio's, neemt de uitvoeringstijd sterk toe wanneer één thread veel elementen moet verwerken (dus bij kleinere thread:elementen-ratio). Wanneer er minder elementen per thread zijn, worden er meer threads gelanceerd en stijgt de parallelisatie, wat resulteert in een kortere uitvoeringstijd.

Bij grotere aantal threads (hogere thread:elementen-ratio) stijgt de uitvoeringstijd opnieuw. Dit komt deels doordat er partiële warps uitgevoerd, maar vooral van de toenemende overhead van het schedulen en beheren van het groot aantal threads en blocks.

In het bijzonder zien we dat de uitvoeringstijd minimaal is bij een ratio van 1/32. Dit komt overeen met de warp-grootte (1 warp = 32 threads), waardoor er volle warps worden uitgevoerd. Er is dus een optimaal punt waarbij de GPU voldoende parallelisme heeft zonder dat de overhead van het aantal threads de uitvoeringstijd vergroot.

3.2 Deel 2: thread divergence

```
Part 2 timings (avg over 500 runs)
RGBRGB: 0.698585 ms
RRGGBB: 0.209092 ms
```

Figuur 3: Terminal output van uitvoeringstijden van gaussiaanse blurring op rode kleurcomponenten

Zoals te zien op figuur 3, is de uitvoeringstijd met van de implementatie met thread divergence (`RGBRGB`) groter dan die van de implementatie zonder thread divergence (`RRGGBB`).

Dit komt doordat binnen de `RGBRGB`-implementatie de threads binnen de warps verschillende uitvoeringspaden volgen. Threads die een rode component verwerken voeren de moeilijkere bereke-

ning uit van de gaussiaanse blur, terwijl threads die een groene of blauwe component verwerken enkel een inversie moeten berekenen. Dit betekent dat de threads van de warp die de groene of blauwe componenten verwerken moeten wachten op de threads die de rode component berekenen, wat zorgt voor een serialisatie en dus minder effectieve parallelisatie.

In de `RRGGBB`-implementatie voeren alle threads binnen een warp dezelfde berekeningen uit, wat thread divergence vermijdt.

De resultaten tonen duidelijk aan dat thread divergence een negatieve impact heeft op de uitvoeringstijd van een kernel.

4 Conclusie

Uit dit labo blijkt dat de prestaties van een GPU sterk afhankelijk is van het kernelontwerp. In het eerste deel zagen we dat een optimale thread:elementen-ratio zorgt voor maximale parallelistatie met minimale overhead.
In het tweede deel toonden de resultaten aan dat thread divergence de uitvoeringstijd verhoogt doordat threads binnen een warp verschillende uitvoeringspaden volgen. Wanneer alle threads binnen een warp dezelfde berekeningen uitvoeren wordt de GPU effficiënter benut.
Een doordachte organisatie van threads en data is dus essentieel voor optimale GPU prestaties.

Referenties

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<extension> <name>.
    ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length

// CUDA kernels

__global__ void invert_image(uint8_t *in, uint8_t *out, int n) {
    // calculate x = 255 -x, ... for each color component - RGB
    // 1 thread per pixelkleurwaarde
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        out[idx] = (255- in[idx]);
    }
}

__global__ void add_1_stride(int* in, int n) {
    // short 1D sequence, adding 1 to each element
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = tid; i < n; i += stride) {
        in[i] += 1;
    }
}

__global__ void invert_strided(uint8_t *in, uint8_t *out, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = tid; i < n; i += stride) {
        out[i] = 255 - in[i];
    }
}

/* Vermijdt divergence
__global__ void R_1D_gaussian(uint8_t *in, uint8_t *out, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    int n_pixels = n/3;
```

```
    for (int p = tid; p < n_pixels; p += stride) {
        //randgevallen: clampen
        int p_m2 = (p - 2 < 0) ? 0 : (p - 2);
        int p_m1 = (p - 1 < 0) ? 0 : (p - 1);
        int p_p1 = (p + 1 >= n_pixels) ? (n_pixels-1) : (p + 1);
        int p_p2 = (p + 2 >= n_pixels) ? (n_pixels-1) : (p + 2);

        int i = p*3;
        int i_m2 = 3 * p_m2;
        int i_m1 = 3 * p_m1;
        int i_p1 = 3 * p_p1;
        int i_p2 = 3 * p_p2;

        float r = 0.1 * in[i_m2] + 0.25 * in[i_m1] + 0.5 * in[i] + 0.25 * in[
    i_p1] + 0.1 * in[i_p2];


        r = (r < 0) ? 0 : r;
        r = (r > 255) ? 255 : r;
        out[i] = (uint8_t) r;
        out[i+1] = 255 - in[i+1];
        out[i+2] = 255 - in[i+2];
    }
}
*/

__global__ void R_1D_gaussian(uint8_t *in, uint8_t *out, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    int n_pixels = n/3;
    for (int idx = tid; idx < n; idx += stride) {
        int c = idx % 3; //=0: rode pixel
        int p = idx/3;
        if (c == 0) {
            //randgevallen: clampen
            int p_m2 = (p - 2 < 0) ? 0 : (p - 2);
            int p_m1 = (p - 1 < 0) ? 0 : (p - 1);
            int p_p1 = (p + 1 >= n_pixels) ? (n_pixels-1) : (p + 1);
            int p_p2 = (p + 2 >= n_pixels) ? (n_pixels-1) : (p + 2);

            int i = p*3;
            int i_m2 = 3 * p_m2;
            int i_m1 = 3 * p_m1;
            int i_p1 = 3 * p_p1;
            int i_p2 = 3 * p_p2;

            float r = 0.1 * in[i_m2] + 0.25 * in[i_m1] + 0.5 * in[i] + 0.25 *
    in[i_p1] + 0.1 * in[i_p2];

            r = (r < 0) ? 0 : r;
            r = (r > 255) ? 255 : r;
```

```
            out[i] = (uint8_t) r;
        } else {
            out[idx] = 255 - in[idx];
        }
    }
}

__global__ void R_1D_gaussian_rrggbb(uint8_t *in_rrggbb, uint8_t *out_rrggbb, int n) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    int n_pixels = n/3;

    //uint8_t* Rin = in_rrggbb;
    uint8_t* Gin = in_rrggbb + n_pixels;
    uint8_t* Bin = in_rrggbb + 2*n_pixels;

    uint8_t* Rout = out_rrggbb;
    uint8_t* Gout = out_rrggbb + n_pixels;
    uint8_t* Bout = out_rrggbb + 2*n_pixels;
    for (int p = tid; p < n_pixels; p += stride) {
        int p_m2 = (p - 2 < 0) ? 0 : (p - 2);
        int p_m1 = (p - 1 < 0) ? 0 : (p - 1);
        int p_p1 = (p + 1 >= n_pixels) ? (n_pixels-1) : (p + 1);
        int p_p2 = (p + 2 >= n_pixels) ? (n_pixels-1) : (p + 2);

        float r = 0.1 * in_rrggbb[p_m2] + 0.25 * in_rrggbb[p_m1] + 0.5 * in_rrggbb[p] + 0.25 * in_rrggbb[p_p1] + 0.1 * in_rrggbb[p_p2];

        r = (r < 0) ? 0 : r;
        r = (r > 255) ? 255 : r;

        Rout[p] = (uint8_t) r;
        Gout[p] = 255 - Gin[p];
        Bout[p] = 255 - Bin[p];
    }
}

uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
     *
```

```
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    //imageFile=fopen("./milamarie.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile
    );

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array){
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./output_image.ppm","wb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile,"P6\n");               // P6 filetype
    fprintf(imageFile,"%d %d\n", M, N);      // dimensions
    fprintf(imageFile,"255\n");              // Max pixel

    // Write the image
    fwrite(image_array, 1, M*N*C, imageFile);

    // Close the file
    fclose(imageFile);
```

```
197 }
198
199 void RGBRGB_to_RRGGBB(uint8_t* rgbrgb, uint8_t* rrggbb, int n_pixels) {
200     uint8_t* R = rrggbb;
201     uint8_t* G = rrggbb + n_pixels;
202     uint8_t* B = rrggbb + 2*n_pixels;
203
204     for (int p = 0; p < n_pixels; ++p) {
205         R[p] = rgbrgb[3*p + 0];
206         G[p] = rgbrgb[3*p + 1];
207         B[p] = rgbrgb[3*p + 2];
208     }
209 }
210
211 void RRGGBB_to_RGBRGB(uint8_t* rrggbb, uint8_t* rgbrgb, int n_pixels) {
212     uint8_t* R = rrggbb;
213     uint8_t* G = rrggbb + n_pixels;
214     uint8_t* B = rrggbb + 2*n_pixels;
215
216     for (int p = 0; p < n_pixels; ++p) {
217         rgbrgb[3*p + 0] = R[p];
218         rgbrgb[3*p + 1] = G[p];
219         rgbrgb[3*p + 2] = B[p];
220     }
221 }
222
223
224 int main (void) {
225     std::string command = "part2"; //invert_strided_loop & part2
226
227     const int threadsPerBlock = 256;
228     const int blocks = (M*N*C + threadsPerBlock - 1) / threadsPerBlock;
229     // Read the image
230     uint8_t* image_array = get_image_array();
231
232     // Allocate output
233     uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
234
235     if (command=="grayscale") {
236         // Convert to grayscale using only the red color component
237         for(int i=0; i<M*N*C; i++){
238             new_image_array[i] = image_array[i/3*3];
239         }
240     }
241     else if (command=="invert"){
242         uint8_t *d_in, *d_out;
243         cudaMalloc((void**)&d_in, M*N*C * sizeof(uint8_t));
244         cudaMalloc((void**)&d_out, M*N*C * sizeof(uint8_t));
245
246         cudaMemcpy(d_in, image_array, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);
```

```
247
248         //Timing
249         cudaEvent_t start_invert, stop_invert;
250         cudaEventCreate(&start_invert);
251         cudaEventCreate(&stop_invert);
252         cudaEventRecord(start_invert);
253
254         // run kernel to invert colours
255         invert_image<<<blocks, threadsPerBlock>>>(d_in, d_out, M*N*C);
256
257         // stop timing
258         cudaDeviceSynchronize();
259         cudaEventRecord(stop_invert);
260         cudaEventSynchronize(stop_invert);
261         float ms_invert;
262         cudaEventElapsedTime(&ms_invert, start_invert, stop_invert);
263
264         cudaMemcpy(new_image_array, d_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);
265
266         cudaFree(d_in);
267         cudaFree(d_out);
268
269         save_image_array(new_image_array);
270
271         std::cout << "invert: "  << ms_invert << " ms" <<std::endl;
272
273     } else if (command=="add1stride") {
274         int NN = 16;
275         int a[NN];
276         int* d_a;
277
278         for (int i = 0; i < NN; i++) {
279             a[i] = i;
280         }
281
282         std::cout << "a before: ";
283         for (int i = 0; i < NN; i++) {
284             std::cout << a[i] << " ";
285         }
286         std::cout << std::endl;
287
288         cudaMalloc((void**)&d_a, NN * sizeof(int));
289         cudaMemcpy(d_a, a, NN * sizeof(int), cudaMemcpyHostToDevice);
290
291         add_1_stride<<<1,12>>>(d_a, NN);
292
293         cudaMemcpy(a, d_a, NN * sizeof(int), cudaMemcpyDeviceToHost);
294
295         std::cout << "a after: ";
296         for (int i = 0; i < NN; i++) {
```

```
            std::cout << a[i] << " ";
        }
        std::cout << std::endl;
        cudaFree(d_a);
    } else if (command=="invert_strided") {
        uint8_t *d_in, *d_out;
        cudaMalloc((void**)&d_in, M*N*C * sizeof(uint8_t));
        cudaMalloc((void**)&d_out, M*N*C * sizeof(uint8_t));

        cudaMemcpy(d_in, image_array, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);

        //Timing
        cudaEvent_t start_invert_strided, stop_invert_strided;
        cudaEventCreate(&start_invert_strided);
        cudaEventCreate(&stop_invert_strided);

        int total_threads = (M*N*C + 15) / 16;
        //threadsPerBlock already defined
        int blocks_strided = (total_threads + threadsPerBlock - 1) /
    threadsPerBlock;
        if (blocks_strided < 1) blocks_strided = 1;

        // run kernel to invert colours
        cudaEventRecord(start_invert_strided);

        invert_strided<<<blocks_strided, threadsPerBlock>>>(d_in, d_out, M*N*C)
    ;

        // stop timing
        cudaDeviceSynchronize();
        cudaEventRecord(stop_invert_strided);
        cudaEventSynchronize(stop_invert_strided);
        float ms_invert_strided;
        cudaEventElapsedTime(&ms_invert_strided, start_invert_strided,
    stop_invert_strided);

        cudaMemcpy(new_image_array, d_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

        cudaFree(d_in);
        cudaFree(d_out);

        save_image_array(new_image_array);

        std::cout << "invert strided: "  << ms_invert_strided << " ms" <<std::
    endl;
    } else if (command == "invert_strided_loop") {

        uint8_t *d_in, *d_out;
        cudaMalloc((void**)&d_in, M*N*C * sizeof(uint8_t));
```

```
342         cudaMalloc((void**)&d_out, M*N*C * sizeof(uint8_t));
343 
344         cudaMemcpy(d_in, image_array, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);
345 
346         //Timing
347         cudaEvent_t start_invert_strided, stop_invert_strided;
348         cudaEventCreate(&start_invert_strided);
349         cudaEventCreate(&stop_invert_strided);
350 
351         int repeats = 500; // sample size
352 
353         //first GPU launch
354         invert_strided<<<1, threadsPerBlock>>>(d_in, d_out, N*M*C);
355         cudaDeviceSynchronize();
356 
357         //progressively use more threads
358         int ratios[] = {4096, 2048, 1024, 512, 256, 128, 64, 32, 16, 8, 4, 2,
    1};
359         int num_ratios = 13;
360         std::cout << "# elements: " << M*N*C << std::endl;
361         std::cout << "threadsPerBlock: " << threadsPerBlock << ", repeats (
    sample size): " << repeats << std::endl;
362         std::cout << "ratio(threads/elements)\tblocks\tthreads\tavg_ms" << std
    ::endl;
363 
364         for (int i = 0; i < num_ratios; i++) {
365             int r = ratios[i];
366             int total_threads = (M*N*C + r - 1) / r;
367             //threadsPerBlock already defined
368             int blocks_strided = (total_threads + threadsPerBlock - 1) /
    threadsPerBlock;
369             if (blocks_strided < 1) blocks_strided = 1;
370             float total_ms = 0.0f;
371 
372             for (int k = 0; k < repeats; k++) {
373                 cudaEventRecord(start_invert_strided);
374 
375                 // run kernel to invert colours
376                 invert_strided<<<blocks_strided, threadsPerBlock>>>(d_in, d_out
    , M*N*C);
377 
378                 // stop timing
379                 //cudaDeviceSynchronize();
380                 cudaEventRecord(stop_invert_strided);
381                 cudaEventSynchronize(stop_invert_strided);
382                 float ms_invert_strided;
383                 cudaEventElapsedTime(&ms_invert_strided, start_invert_strided,
    stop_invert_strided);
384                 total_ms += ms_invert_strided;
385             }
```

```
386             float avg_ms = total_ms/repeats;
387 
388             std::cout << "1:" << r
389                   << "\t\t\t" << blocks_strided
390                   << "\t" << (blocks_strided * threadsPerBlock)
391                   << "\t" << avg_ms << "\n";
392         }
393 
394         cudaMemcpy(new_image_array, d_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);
395 
396         cudaFree(d_in);
397         cudaFree(d_out);
398 
399         save_image_array(new_image_array);
400     } else if (command == "R_1D_gaussian") {
401         uint8_t *d_in, *d_out;
402         cudaMalloc((void**)&d_in, M*N*C * sizeof(uint8_t));
403         cudaMalloc((void**)&d_out, M*N*C * sizeof(uint8_t));
404 
405         cudaMemcpy(d_in, image_array, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);
406 
407         //Timing
408         cudaEvent_t start_r_1d_gaussian, stop_r_1d_gaussian;
409         cudaEventCreate(&start_r_1d_gaussian);
410         cudaEventCreate(&stop_r_1d_gaussian);
411 
412         int total_threads = (M*N*C + 15) / 16;
413         //threadsPerBlock already defined
414         int blocks_strided = (total_threads + threadsPerBlock - 1) /
    threadsPerBlock;
415         if (blocks_strided < 1) blocks_strided = 1;
416 
417         // run kernel to invert colours
418         cudaEventRecord(start_r_1d_gaussian);
419 
420         R_1D_gaussian<<<blocks_strided, threadsPerBlock>>>(d_in, d_out, M*N*C);
421 
422         // stop timing
423         cudaDeviceSynchronize();
424         cudaEventRecord(stop_r_1d_gaussian);
425         cudaEventSynchronize(stop_r_1d_gaussian);
426         float ms_r_1d_gaussian;
427         cudaEventElapsedTime(&ms_r_1d_gaussian, start_r_1d_gaussian,
    stop_r_1d_gaussian);
428 
429         cudaMemcpy(new_image_array, d_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);
430 
431         cudaFree(d_in);
```

```
432         cudaFree(d_out);
433 
434         save_image_array(new_image_array);
435 
436         std::cout << "R_1D_gaussian: "  << ms_r_1d_gaussian << " ms" <<std::
    endl;
437     } else if (command == "rrggbb") {
438         uint8_t *d_in, *d_out;
439         cudaMalloc((void**)&d_in, M*N*C * sizeof(uint8_t));
440         cudaMalloc((void**)&d_out, M*N*C * sizeof(uint8_t));
441 
442         uint8_t* image_array_rrggbb = (uint8_t*)malloc(M*N*C);
443         uint8_t* image_array_rrggbb_out = (uint8_t*)malloc(M*N*C);
444 
445         RGBRGB_to_RRGGBB(image_array, image_array_rrggbb, M*N*C/3);
446 
447         cudaMemcpy(d_in, image_array_rrggbb, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);
448 
449         //Timing
450         cudaEvent_t start_rrggbb, stop_rrggbb;
451         cudaEventCreate(&start_rrggbb);
452         cudaEventCreate(&stop_rrggbb);
453 
454 
455         int total_threads = (M*N*C/3 + 15) / 16;
456         //threadsPerBlock already defined
457         int blocks_rrggbb = (total_threads + threadsPerBlock - 1) /
    threadsPerBlock;
458         if (blocks_rrggbb < 1) blocks_rrggbb = 1;
459 
460         // run kernel to invert colours
461         cudaEventRecord(start_rrggbb);
462 
463         R_1D_gaussian_rrggbb<<<blocks_rrggbb, threadsPerBlock>>>(d_in, d_out, M
    *N*C);
464 
465         // stop timing
466         cudaDeviceSynchronize();
467         cudaEventRecord(stop_rrggbb);
468         cudaEventSynchronize(stop_rrggbb);
469         float ms_rrggbb;
470         cudaEventElapsedTime(&ms_rrggbb, start_rrggbb, stop_rrggbb);
471 
472         cudaMemcpy(image_array_rrggbb_out, d_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);
473 
474         cudaFree(d_in);
475         cudaFree(d_out);
476 
477         RRGGBB_to_RGBRGB(image_array_rrggbb_out, new_image_array, M*N*C/3);
```

```
478
479         save_image_array(new_image_array);
480
481         std::cout << "rrggbb: "  << ms_rrggbb << " ms" <<std::endl;
482     } else if (command == "part2") {
483         int repeats = 500;
484
485         uint8_t* d_rgb_in,* d_rgb_out,* d_rr_in,* d_rr_out;
486
487         cudaMalloc((void**)&d_rgb_in, M*N*C * sizeof(uint8_t));
488         cudaMalloc((void**)&d_rgb_out, M*N*C * sizeof(uint8_t));
489         cudaMalloc((void**)&d_rr_in, M*N*C * sizeof(uint8_t));
490         cudaMalloc((void**)&d_rr_out, M*N*C * sizeof(uint8_t));
491
492         uint8_t* image_array_rr_in = (uint8_t*)malloc(M*N*C);
493         uint8_t* image_array_rr_out = (uint8_t*)malloc(M*N*C);
494
495         RGBRGB_to_RRGGBB(image_array, image_array_rr_in, M*N*C/3);
496
497         cudaMemcpy(d_rgb_in, image_array, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);
498         cudaMemcpy(d_rr_in, image_array_rr_in, M*N*C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);
499
500         int r = 16; // pixels per thread
501         int total_threads_rrggbb = (M*N*C/3 + r - 1)/r;
502         int blocks_rrggbb = (total_threads_rrggbb + threadsPerBlock - 1)/
    threadsPerBlock;
503
504         int total_threads_rgb = 3 * total_threads_rrggbb; //gelijke threads per
     pixel voor beide implementaties
505         int blocks_rgb = (total_threads_rgb + threadsPerBlock - 1)/
    threadsPerBlock;
506
507         // warmup
508         R_1D_gaussian<<<blocks_rgb, threadsPerBlock>>>(d_rgb_in, d_rgb_out, M*N
    *C);
509         R_1D_gaussian_rrggbb<<<blocks_rrggbb, threadsPerBlock>>>(d_rr_in,
    d_rr_out, M*N*C);
510         cudaDeviceSynchronize();
511
512         cudaEvent_t start, stop;
513         cudaEventCreate(&start);
514         cudaEventCreate(&stop);
515
516         float ms_total_rgb = 0.0f;
517         for (int k = 0; k < repeats; k++) {
518             cudaEventRecord(start);
519             R_1D_gaussian<<<blocks_rgb, threadsPerBlock>>>(d_rgb_in, d_rgb_out,
     M*N*C);
520             cudaEventRecord(stop);
```

```
            cudaEventSynchronize(stop);

            float ms = 0.0f;
            cudaEventElapsedTime(&ms, start, stop);
            ms_total_rgb += ms;
        }
        float avg_ms_rgb = ms_total_rgb / repeats;

        float ms_total_rr = 0.0f;
        for (int k = 0; k < repeats; k++) {
            cudaEventRecord(start);
            R_1D_gaussian_rrggbb<<<blocks_rrggbb, threadsPerBlock>>>(d_rr_in,
    d_rr_out, M*N*C);
            cudaEventRecord(stop);
            cudaEventSynchronize(stop);

            float ms = 0.0f;
            cudaEventElapsedTime(&ms, start, stop);
            ms_total_rr += ms;
        }
        float avg_ms_rr = ms_total_rr / repeats;

        cudaMemcpy(new_image_array, d_rgb_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);
        //save_image_array(new_image_array);

        cudaMemcpy(image_array_rr_out, d_rr_out, M*N*C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);
        RRGGBB_to_RGBRGB(image_array_rr_out, new_image_array, M*N*C/3);
        save_image_array(new_image_array);

        std::cout << "Part 2 timings (avg over " << repeats << " runs)" << std
    ::endl;
        std::cout << "RGBRGB: " << avg_ms_rgb << " ms" << std::endl;
        std::cout << "RRGGBB: " << avg_ms_rr << " ms" << std::endl;

        cudaFree(d_rgb_in);
        cudaFree(d_rgb_out);
        cudaFree(d_rr_in);
        cudaFree(d_rr_out);

    }

    return 0;
}
```

Codefragment 1: Code labo 3

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN
TECHNOLOGIECAMPUS GENT
Gebroeders De Smetstraat 1
9000 GENT, België
tel. + 32 92 65 86 10
fax + 32 92 25 62 69
iiw.gent@kuleuven.be
www.iiw.kuleuven.be

LID VAN ASSOCIATIE KU LEUVEN